

Dyhia BERKHOUCHE

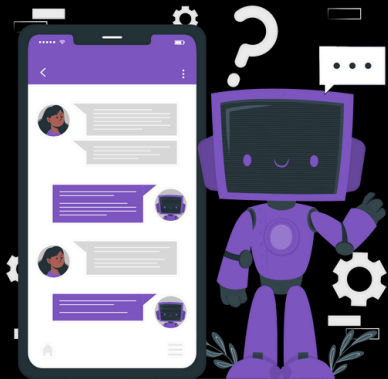
Je vous présente mon projet pour langage Rust

...

◆ **Projet phare :**

Chatbot Phishinguard

- **Sensibilisation aux cyberattaques.**
- **Simplifie la cybersécurité pour les non-professionnels.**
 - **Scénarios interactifs pour détecter le phishing.**



Rust



Phishinguard

Phishinguard, c'est une plateforme applicative que j'ai créée pour aider les entreprises à sensibiliser et protéger leurs employés contre les cyberattaques, en particulier le phishing. Ce type de menace est devenu omniprésent et représente un danger majeur pour les organisations.



Campagnes

SMS



EMAIL



Contacts

dyhia



Simulation



Salah



Lorenzo



sarah



API

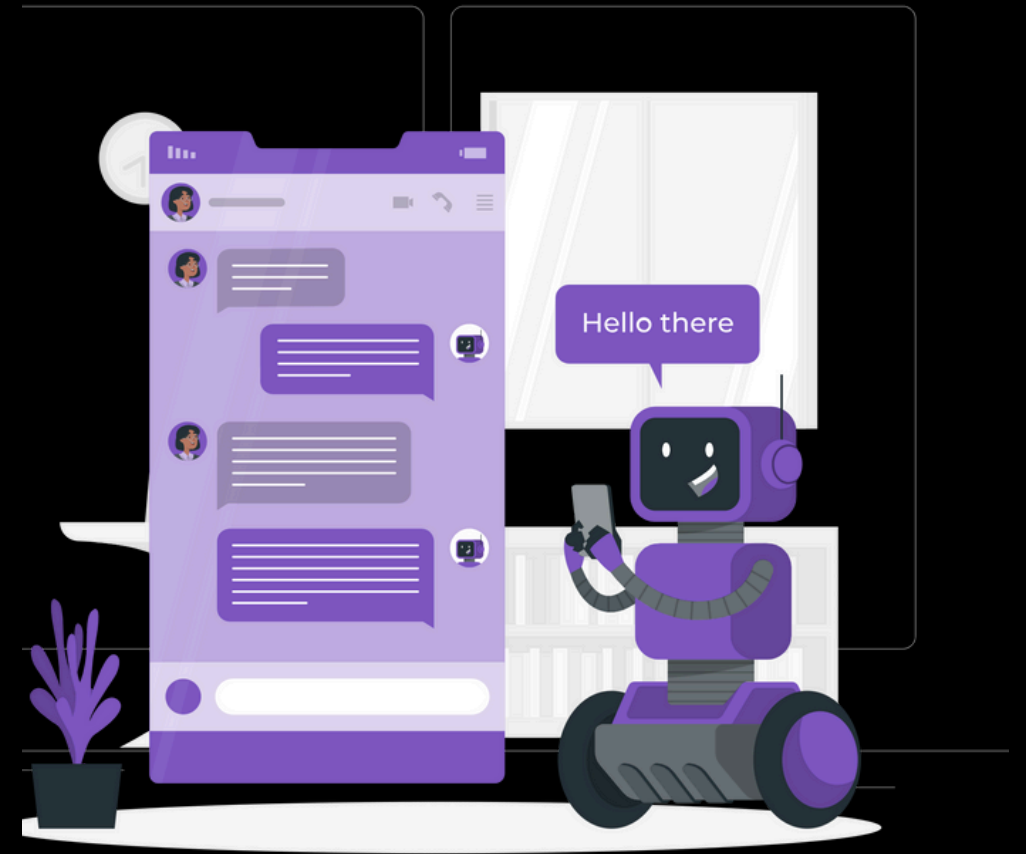
Mailtrap Test - email



Twilio SMS API - sms

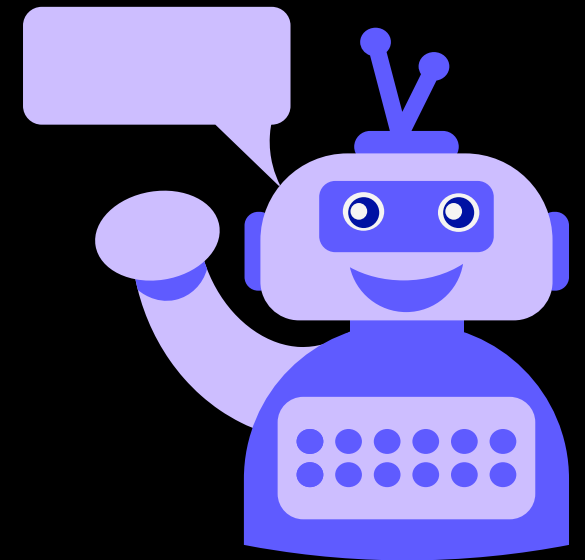


J'ai eu l'idée de développer ce chatbot qui s'intègre à cette plateforme. Son objectif est de simplifier l'accès aux fonctionnalités de Phishinguard, même pour les utilisateurs non techniques. Grâce à cet outil, n'importe quel employé peut apprendre à reconnaître et à éviter les cyberattaques, rendant ainsi la sensibilisation accessible à tous.



Introduction au Chatbot Phishinguard

Le Chatbot Phishinguard est une solution innovante conçue pour renforcer la cybersécurité des entreprises en sensibilisant leurs employés aux menaces de phishing. Aujourd'hui, les cyberattaques, et notamment le phishing, représentent un risque majeur pour les organisations de toutes tailles. Malheureusement, ces menaces exploitent souvent un maillon faible : le facteur humain.



Rôle du chatbot

Le rôle de ce chatbot est de simplifier l'accès aux fonctionnalités de Phishinguard, une plateforme de sensibilisation et de formation, en la rendant accessible même aux utilisateurs non techniques. Il nous permet également de mieux naviguer dans la plateforme et facilite l'utilisation de toutes ses fonctionnalités.



Fonctionnement global

Objectif :

- Implémenter un chatbot qui répond aux messages des utilisateurs en fonction d'un ensemble de paires questions-réponses prédéfinies.
- Ajouter dynamiquement de nouvelles paires question-réponse.



2. Technologies utilisées :

- Warp : Un framework léger pour gérer les routes HTTP.
- Serde : Pour la sérialisation/désérialisation JSON.
- Tokio : Pour exécuter des tâches asynchrones.
- HashMap : Pour stocker dynamiquement les paires questions-réponses.



Bibliothèques Importées :

warp::Filter : Gère les routes HTTP et les filtres pour le serveur.

serde::{Deserialize, Serialize} : Sérialisation/désérialisation des structures en JSON.

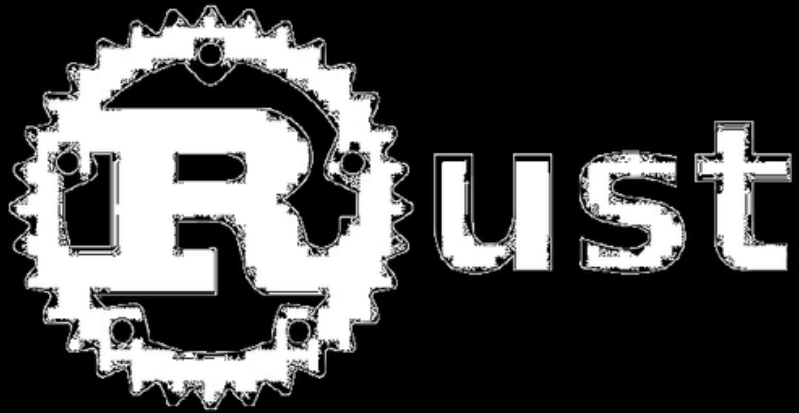
std::sync::{Arc, Mutex} : Gère un état partagé de manière sécurisée entre threads.

std::collections::HashMap : Gère dynamiquement les paires question-réponse.



Structures JSON :

1. ChatRequest : Représente la requête client, contient un message (texte).
2. ChatResponse : Représente la réponse du bot, contient un reply (texte).
3. AddQaRequest : Requête pour ajouter une nouvelle paire question-réponse.



État Partagé :

AppState :

- history : Historique des messages utilisateur/bot.
- qa_pairs : Contient des paires question-réponse (exemple : "hello" → "Bonjour !").

Utilise Arc<Mutex<AppState>> pour un accès sécurisé entre threads.



Routes :

1./chat :

- Méthode : POST.
- Extrait un message JSON utilisateur, recherche une réponse dans qa_pairs.
- Sauvegarde la conversation dans l'historique.
- Renvoie la réponse au client.

2./add_qa :

- Méthode : POST.
- Ajoute dynamiquement une nouvelle paire question-réponse dans qa_pairs.

Middleware CORS :

Permet des requêtes POST depuis n'importe quelle origine avec des en-têtes spécifiques.

j'ai choisi de le mettre au milieu du code pour qu'il soit plus fluide

CORS

Cross Origin Resource Sharing

Fonctions Clés :

handle_chat :

- Gère les messages envoyés à /chat.
- Cherche une réponse dans qa_pairs, ou envoie un message générique en cas d'absence.
- Met à jour l'historique.

add_question_answer :

- Permet d'ajouter dynamiquement une paire question-réponse via /add_qa

with_state :

- Injecte l'état partagé (AppState) dans chaque requête.

query_platform :

- Effectue une requête à une URL externe pour récupérer des informations JSON.

Serveur :

- Lance les routes /chat et /add_qa sur `http://localhost:5501`.
- Intègre le middleware CORS pour les requêtes externes.



Usage Global :

Ce code est conçu pour un chatbot interactif avec :

- Un historique des conversations.
- Une capacité d'apprentissage dynamique via des ajouts de questions-réponses.
- Une ouverture à l'intégration d'APIs externes avec `query_platform`.



Côté Python :

Ajoutez des routes pour fournir les informations nécessaires (/info/contacts, /info/apis, /info/clicks, etc.).

Assurez-vous que le serveur Flask est accessible sur <http://127.0.0.1:5001>.

Côté Rust :

Modifiez `handle_chat` pour interroger la plateforme Python en cas de besoin.
Ajoutez `query_platform` pour faire des requêtes HTTP aux routes Python.

Conclusion



Phishinguard Chatbot

Une solution moderne pour :

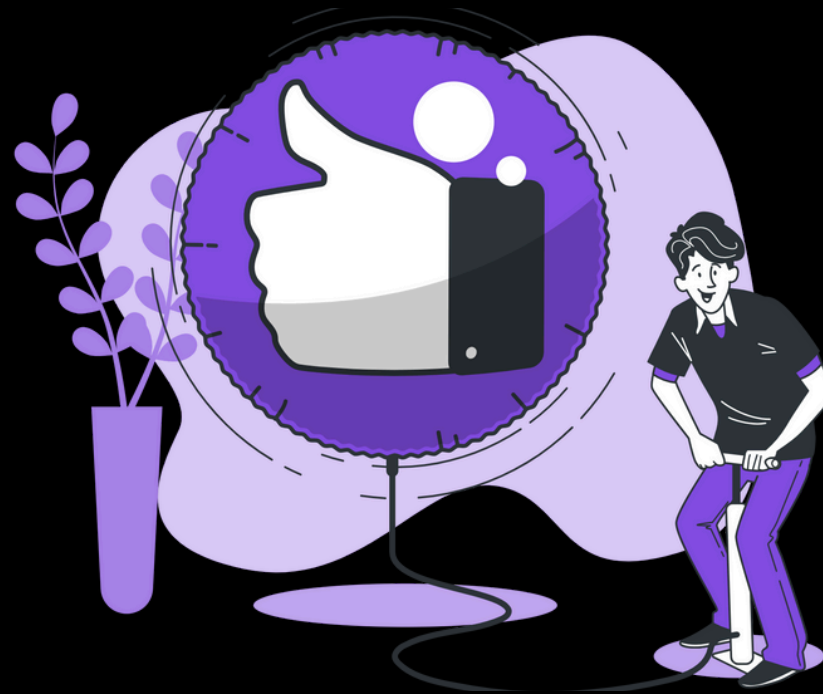
- **Sensibiliser aux cyberattaques, en particulier le phishing.**
- **Rendre la cybersécurité accessible aux non-professionnels.**
 - **Renforcer la vigilance et réduire les risques**



Perspectives d'avenir :

- **Intégration d'intelligence artificielle pour des réponses plus naturelles.**
- **Personnalisation avancée pour répondre aux besoins spécifiques des entreprises.**

Je vous remercie sincèrement pour votre écoute. Si vous avez des questions ou souhaitez discuter plus en détail de certains points, je serai ravi de poursuivre l'échange dans les commentaires.





Lien GitHub:

https://github.com/berkhouchedyhia/pishinguard_chatbot



Liens vidéo YouTube :

<https://youtu.be/ZZAGUcJvD8Y>